UNIT 1: MACHINE LEARNING FOUNDATIONS & SUPERVISED LEARNING

1. Introduction to Supervised Learning
Supervised learning is a subfield of machine learning where the algorithm learns a mapping function from input variables (X) t
- Formal Definition: Given a training dataset D = {(x_1, y_1), (x_2, y_2), ..., (x_n, y_n)}, the objective is to find a hypothesis fun
- Key Components:
  1. Features (Inputs): Attributes that describe data points.
  2. Labels (Outputs): Ground-truth target values assigned to instances.
  3. Loss Function: Measures the discrepancy between predicted and actual labels.
- Real-World Examples:
  - Spam email classification (Input: email text; Output: Spam / Not Spam).
  - House price prediction (Input: area, bedrooms, location; Output: Price in USD).
- Categories of Supervised Learning:
  - Classification: Output is discrete/categorical (e.g., Disease Diagnosis: Positive / Negative).
  - Regression: Output is continuous/numerical (e.g., Stock price prediction, Temperature forecast).

2. Overfitting and Underfitting
- Overfitting: A modeling error where a machine learning model learns the training data too well, capturing noise and random f
  - Prevention Techniques: Regularization (L1 Lasso, L2 Ridge), Cross-Validation (K-Fold), Pruning in Decision Trees, Dropou
- Underfitting: Occurs when a model is too simple to capture the underlying structure of the data, resulting in poor performance
  - Solutions: Increase model complexity, engineer more relevant features, reduce regularization strength.

UNIT 2: ENSEMBLE METHODS & RANDOM FOREST

1. Concept of Ensemble Learning
Ensemble learning is a machine learning paradigm where multiple base models (weak learners) are strategically combined to
- Primary Ensemble Paradigms:
  1. Bagging (Bootstrap Aggregation): Reduces variance by training multiple base models independently on random subsets o
  2. Boosting: Reduces bias by sequentially training base models where each subsequent model focuses on correcting the err
  3. Stacking: Trains a meta-learner to combine predictions of heterogeneous base models.

2. Random Forest Deep Dive
Random Forest is a popular supervised ensemble learning algorithm that constructs a multitude of decision trees during trainin

- How Random Forest Works (Algorithm Steps):
  1. Bootstrap Sampling: Randomly select n samples from the dataset with replacement.
  2. Random Feature Subspace (Feature Bagging): At each split in a decision tree, select a random subset of m features (typic
  3. Tree Construction: Grow deep unpruned decision trees on each bootstrap sample.
  4. Aggregation: For classification, use majority voting; for regression, compute the arithmetic mean of all tree outputs.
- Why Random Forest Reduces Overfitting:
  - Individual decision trees are prone to high variance and overfitting. By training decorrelated trees on different random subse
- Key Advantages:
  - Handles high-dimensional datasets with thousands of input variables.
  - Robust against outliers and missing values.
  - Provides built-in Feature Importance estimation (via Gini Impurity reduction or Permutation Importance).
  - Does not require feature scaling/normalization.
- Real-World Examples:
  - Credit card fraud detection.
  - Customer churn prediction in telecommunications.
  - Medical diagnosis from gene expression data.

UNIT 3: DEEP LEARNING & CONVOLUTIONAL NEURAL NETWORKS (CNN)

1. Convolutional Neural Networks (CNN) Architecture
Convolutional Neural Networks (CNNs) are specialized deep feed-forward neural networks designed for processing grid-struct
- Core Layers of a CNN:
  1. Convolutional Layer: Applies a set of learnable spatial filters (kernels) that slide across the input feature map performing el
     - Hyperparameters: Kernel size (e.g., 3x3, 5x5), Stride (step size of filter movement), Padding (Valid / Same to control spat
  2. Activation Function (ReLU): Introduces non-linearity to the network (f(x) = max(0, x)), speeding up convergence and mitiga
  3. Pooling Layer (Subsampling): Downsamples spatial dimensions (height and width) to reduce computational complexity an
     - Common Types: Max Pooling (takes maximum value in window), Average Pooling (computes average value).
  4. Fully Connected (Dense) Layer: Flattens the 2D/3D feature maps into a 1D vector and connects every neuron to all activat
  5. Softmax Output Layer: Converts raw logits into normalized class probability distributions.
- Key Advantages over Standard Multi-Layer Perceptrons (MLP):
  - Parameter Sharing: The same filter weights are used across the entire image.
  - Sparse Connectivity: Neurons connect only to a local receptive field rather than all inputs.
  - Translation Invariance: Can recognize features regardless of their spatial position in the image.

UNIT 4: DATA STRUCTURES & ALGORITHMS · BFS VS DFS

1. Breadth-First Search (BFS)
- Traversal Mechanism: Explores graph or tree level by level, visiting all neighbor vertices at the current depth before moving d
- Underlying Data Structure: Queue (First In, First Out - FIFO).
- Time Complexity: O(V + E) where V is vertices and E is edges.
- Space Complexity: O(V) due to storing all nodes at the current level.
- Optimal For: Finding the shortest path in unweighted graphs, finding connected components, level-order traversal.

2. Depth-First Search (DFS)
- Traversal Mechanism: Explores as deep as possible along each branch before backtracking.
- Underlying Data Structure: Stack (Last In, First Out - LIFO) or recursion call stack.
- Time Complexity: O(V + E).
- Space Complexity: O(H) where H is the maximum tree height / recursion depth (O(V) in worst case).
- Optimal For: Topological sorting, cycle detection in directed/undirected graphs, solving maze puzzles, pathfinding.

3. Key Comparison Table (BFS vs DFS):
Metric	Breadth-First Search (BFS)	Depth-First Search (DFS)
Traversal Strategy	Level-by-level (horizontal)	Depth-first along branches (vertical)
Data Structure	Queue (FIFO)	Stack (LIFO) / Recursion
Shortest Path Guarantee	Guaranteed for unweighted graphs	Not guaranteed
Memory Requirement	Higher (stores entire level frontier)	Lower (stores single path from root)
Backtracking Needed	No	Yes